Project 1: Linear Regression and Regularization

(First discussion: Sept 23; Last questions: Oct 7; Deadline: Oct 14; In charge: Zhexin Wu)

This project explores linear regression and regularization techniques in the context of predicting house prices. You will use the *Housing* dataset from Ames, Iowa (USA), which contains 1,460 observations of residential properties. The dataset includes one target variable, *SalePrice* (the sale price of the house in U.S. dollars), and 79 explanatory variables describing different aspects of the homes. Of these predictors, 35 are numerical and 44 are categorical.

The goal of the project is to build models that both explain and predict *SalePrice*, while comparing the strengths and limitations of different regression approaches. You can refer to the file `data_description.txt` for a detailed description of all variables.

This dataset is widely used in predictive modeling competitions and mimics real-world challenges such as skewed target distributions, missing values, multicollinearity among predictors, and many categorical features. Addressing these issues is part of the learning objective of the project.

Submission and report. Submit one fully executed Jupyter notebook. At the end of the notebook, include a concise report in Markdown cells under the heading *Project Report*. The report should summarize your approach, model selection, key results, interpretation, robustness, and limitations. Detailed computations and intermediate results should remain in the main body of the notebook.

Code quality. Keep your analysis readable and reproducible. Avoid unnecessary duplication and use small reusable helper functions where appropriate. Classes or object-oriented design are not required.
Grading. The implementation and results account for 80% of the project grade, and the Project Report accounts for 20%.

1. To start, you need to import the dataset into Python in the correct format.

   a) Import the dataset `Housing.csv` in Python as a pandas DataFrame. Separate the explanatory features X (*Housing*) from the target variable y (*SalePrice*).

   b) Graphically determine whether the target variable *SalePrice* is approximately Gaussian. If not, suggest a suitable transformation to bring *SalePrice* closer to a Gaussian distribution and apply this transformation to the dataset. Why is it important to consider such potential transformations?

   c) Since the regression models have to be evaluated on a different dataset than the one used for training, split the data into two subsets: $(X, y)_{train}$ and $(X, y)_{test}$. Randomly assign 70% of the observations to the training set and the remaining 30% to the test set. Keep an unprocessed copy of these training and test splits, since you will return to the raw features in Question 3 when constructing a cross-validation pipeline.

   d) Replace missing values in X using the training data statistics only:
      - For numerical features, replace missing values with the mean of the column (computed from the training set).
      - For categorical features, replace missing values with the most frequent category in the corresponding column (computed from the training set).

– Some categorical variables admit `'NA'` (or `'None'`) as a valid category, which should be treated as an actual level and not as missing.

Next, standardize all numerical features by applying a z-score transform using the training mean and standard deviation, and apply the same transformation (using these training set statistics) to the test data. Finally, use one-hot encoding for all categorical features (e.g., with `pd.get_dummies`). After encoding, ensure that the test set has the same number of columns as the training set and the features appear in the correct order.

Hint: If `pd.get_dummies` is applied separately to the training and test sets, they may produce different dummy-variable columns because some categorical levels occur in only one of the two sets. The regression model must receive the same feature columns at training and prediction time. You can use

```
X_test = X_test.reindex(columns=X_train.columns, fill_value=0)
```

to make the encoded test columns match the training columns and their order. Dummy variables corresponding to training categories that are absent from the test set are then filled with zeros.

2. This question focuses on building a linear regression model to predict the variable *SalePrice* using only the 35 numerical features.

   a) Restrict your analysis to the numerical predictors only (i.e., exclude all categorical features from X). Fit a linear regression model on the training dataset using the `sklearn` Python package. The regression should include an intercept term. Present a table with the regression coefficients for each feature. Compare the in-sample and out-of-sample Mean Squared Error (MSE) and R^2.

   If you transformed the target variable, you have to inverse-transform the predictions before computing MSE and R^2, so that these metrics are reported on the original *SalePrice* scale. In addition, also report the MSE and R^2 on the transformed scale, and comment on the differences between the two. What does each set of metrics tell you about model performance?

   b) The `sklearn` package does not provide standard errors for the estimated regression coefficients, which are essential tools to assess the statistical precision of an estimate.[1] Therefore, you will now use matrix algebra in Python with the `numpy` package to compute the standard errors of the estimated coefficients $\hat{\beta}$. All computations in this part should be performed using the training set only. Let $A \in \mathbb{R}^{m\times(d+1)}$ denote the design matrix (including a column of ones for the intercept term), and let $y \in \mathbb{R}^m$ denote the observed target values.

      (i) Compute the estimated coefficients $\hat{\beta}$ using

      $$\hat{\beta} = (A^\top A)^{-1} A^\top y\,.$$

      Note that $\hat{\beta}_0$ denotes the estimate of the intercept.

      Practical note: In code, it is better not to form matrix inverses explicitly. Instead, if the system is numerically well behaved, one can use `np.linalg.solve`. Either way, solve the normal equations here via `np.linalg.solve(A.T @ A, A.T @ y)`.

[1] Under the classical linear model with homoscedastic, normally distributed errors, the OLS estimator $\hat{\beta}$ is exactly normal, and each t-statistic $\hat{\beta}_j / \mathrm{SE}(\hat{\beta}_j)$ follows a t-distribution with $m-(d+1)$ degrees of freedom. This enables confidence intervals and significance tests.

(ii) Compute the standard error of each coefficient $\hat{\beta}_j$, $j = 0, \ldots, d$, using

$$\widehat{\sigma}^2 = \frac{1}{m-(d+1)} \sum_{i=1}^{m} (y_i - \hat{y}_i)^2,$$

and

$$\mathrm{SE}(\hat{\beta}_j) = \sqrt{\widehat{\sigma}^2 \cdot \left[(A^\top A)^{-1}\right]_{jj}},$$

where $[(A^\top A)^{-1}]_{jj}$ denotes the j-th diagonal element. Recall that $j = 0$ corresponds to the intercept.

(iii) Using your matrix-algebra implementation, verify that the in-sample MSE and R^2 agree with the corresponding results from Question 2.a), up to numerical rounding.

(iv) Inspect the numerical structure of the design matrix A. Compute its rank and singular values, and comment on whether A is full column rank and whether the smallest singular values indicate potential numerical instability. Relate your findings to the OLS calculations above.

(v) Replace $(A^\top A)^{-1}A^\top$ by the Moore–Penrose pseudoinverse A^+ (use `np.linalg.pinv` with its default cutoff). Do $\hat{\beta}$, $\widehat{\sigma}^2$, and the standard errors change? Briefly explain when the results are identical and when they can differ, using your findings from part (iv).

(vi) Confirm your results using the OLS function from the `statsmodels` package. Report the coefficient table and standard errors, and check that they match your matrix-algebra results up to numerical rounding. If they differ, relate the discrepancy to the rank and numerical-stability diagnostics above.

3. In this question, you will implement regularization techniques and compare their performance to ordinary least squares. Use the same training and test split as before to ensure consistency across questions. In Question 3.a), use the prepared *Housing* dataset from Question 1. For the cross-validation analysis in Question 3.b), return to the unprocessed training and test features saved in Question 1.c).

   In this project, all learned preprocessing steps used during cross-validation should be fitted within the corresponding cross-validation training fold. This convention allows you to construct the complete model-selection procedure using `Pipeline` and `GridSearchCV`.

   a) Fit an OLS regression of the potentially transformed target variable *SalePrice* on all explanatory variables in the prepared *Housing* dataset. Report the in-sample and out-of-sample MSE and R^2 on the original *SalePrice* scale. If the target was transformed, inverse-transform the predictions before computing these metrics.

   Compare these results with the regression using only numerical features from Question 2.a). Does including the categorical features improve out-of-sample predictive performance? How does the gap between in-sample and out-of-sample performance change?

   b) Implement the Truncated Pseudoinverse, Ridge, Lasso, and Elastic Net regressions. Use 8-fold cross-validation on the training set to tune the hyperparameters of each method, using MSE as the selection criterion. Use the same cross-validation folds for all methods.

   For this part, construct the complete model-selection workflow using `sklearn`'s

   - `ColumnTransformer`,
   - `Pipeline`, and
   - `GridSearchCV`.

   A minimal example of this workflow is provided in the project template. Follow the steps below.

(i) Start from the unprocessed training and test features saved in Question 1.c). Construct a preprocessing component that reproduces Question 1.d), including imputation, standardization, one-hot encoding, and the special treatment of `'NA'` / `'None'` when these are valid categorical levels.
Place this preprocessing component inside the pipeline so that its learned quantities are fitted separately using the training observations of each cross-validation fold. Make sure that categories not observed in a particular training fold can still be handled when transforming the corresponding validation fold.

(ii) Construct one pipeline for each regularization method, using the same preprocessing component.
For Ridge, Lasso, and Elastic Net, place the regression model directly after the preprocessor.
For the Truncated Pseudoinverse model, insert `TruncatedSVD` after the preprocessor and before `LinearRegression`.

(iii) Specify reasonable hyperparameter grids and tune them with `GridSearchCV`. In particular, consider:
- the regularization strength for Ridge and Lasso;
- the regularization strength and mixing parameter for Elastic Net; and
- the number of retained singular directions for the Truncated Pseudoinverse model.

Pipeline parameters follow the usual naming convention. For example, `model__alpha` refers to the parameter `alpha` of the pipeline step named `model`. For the Truncated Pseudoinverse model, tune the number of retained singular directions through `svd__n_components`. This hyperparameter specifies the number of retained singular components rather than a singular-value cutoff.

(iv) Inspect whether the selected hyperparameter values lie at or near the boundary of the corresponding search grids. If so, extend the relevant range and repeat the search. Briefly justify your final hyperparameter ranges.

(v) Use the selected and refitted models to compute in-sample and out-of-sample MSE and R^2 on the original *SalePrice* scale. If the target was transformed, inverse-transform the predictions before computing these metrics.
Compare the results with the OLS models from Questions 2.a) and 3.a).

Summarize your results in a table. Include the four regularized models as well as the OLS benchmarks from Questions 2.a) and 3.a). For quantities that do not apply to an OLS benchmark, use "–". The table should report:

- the selected hyperparameter(s);
- for each regularized model, the mean cross-validation MSE across the 8 validation folds for the selected hyperparameter configuration, together with its standard deviation;
- the in-sample MSE and R^2 on the original *SalePrice* scale; and
- the out-of-sample MSE and R^2 on the original *SalePrice* scale.

If *SalePrice* was transformed, the cross-validation MSE used for model selection is computed on the transformed target scale.

Hint: For `GridSearchCV`, the cross-validation quantities for the selected hyperparameter configuration can be obtained from the selected row of `cv_results_`, for example using `best_index_`, `mean_test_score`, and `std_test_score`. If you use `neg_mean_squared_error` as the scoring rule, remember to reverse the sign of `mean_test_score` when reporting MSE.
All regressions should include an intercept term. The intercept must not be penalized during regularization. Why is it important not to penalize the intercept?

c) Explain the meaning of the pipeline parameter notation (e.g., `model__alpha`) and what `refit=True` in `GridSearchCV` does after the best hyperparameters have been selected. Why is this refitting step useful before the final evaluation on the held-out test set?

d) For the Lasso and Elastic Net regularization techniques, how many estimated feature coefficients are non-zero ($\hat{\beta}_j \neq 0$), excluding the intercept?

Compare these results with Ridge, which generally shrinks coefficients without setting them exactly to zero.

For the Truncated Pseudoinverse model, report the selected number of retained singular directions instead of counting non-zero regression coefficients. Explain why truncating singular directions is conceptually different from coefficient sparsity in Lasso or Elastic Net.

e) Based on your findings from Questions 2 and 3, which model would you recommend for predicting house prices? Justify your choice not only by comparing performance metrics, but also by discussing the nature of the problem (e.g., number of features, presence of categorical variables, potential collinearity, sparsity, nonlinearity). Explain how the strengths and limitations of the chosen method align with the problem structure.